(고) 알고리즘과 프로그래밍 수업 모형 구성

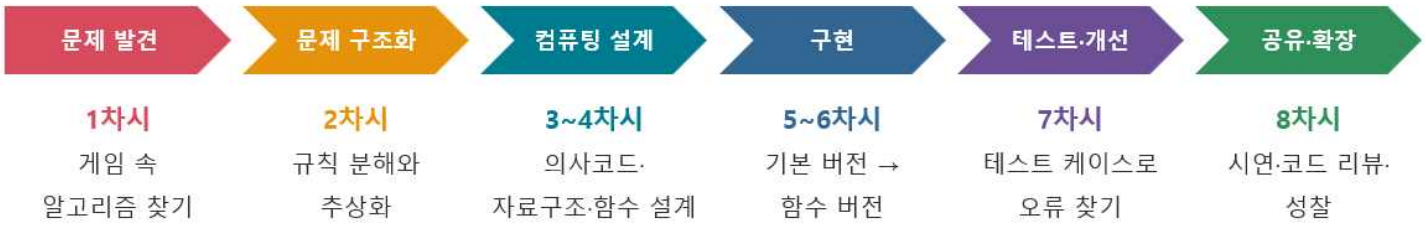
김해수남고 김동균

Ⅰ. 교육과정 분석

프로젝트 주제	**행맨 게임 알고리즘을 구현해보며 문제 해결 능력 키우기**		
주요 교과(영역)	정보(알고리즘과 프로그래밍)	**대상**	1학년
연계 교과(영역) (필요한 경우)	수학(중학교 확률 · 공통수학1 경우의 수) - 확률: 단어 목록에서 각 알파벳이 들어 있는 단어의 비율을 딕셔너리로 세어 '먼저 고를 글자'의 성공 확률 비교하기(예: 동물 단어 64개 중 a 50.0%, x 1.6%) - 영어 - 어휘: 주제별 영단어와 뜻을 딕셔너리({단어: 뜻})로 정리하여 뜻 힌트·주제 선택 기능 만들기		
성취 기준	[12정보03-01] 복잡한 문제를 해결 가능한 작은 문제로 분해하고 모델링한다. [12정보03-06] 다차원 데이터 구조를 활용한 프로그램을 작성한다. [12정보03-07] 다양한 제어 구조를 복합적으로 활용한 프로그램을 작성한다. (연계) [12정보03-09] 실생활 및 다양한 학문 분야의 문제 해결을 위한 프로그램을 협력적으로 설계·구현한다. (연계) [12정보03-10] 문제 해결을 위한 프로그램의 성능을 평가하고 공유한다.		
경남형 AI 디지털 역량	AI·디지털 자기주도 역량 AI·디지털 문제해결 역량 AI·디지털 협력적 소통 역량		
핵심 아이디어	• 복잡한 문제는 해결 가능한 작은 문제로 분해하고, 핵심 요소만 남기는 추상화를 통해 컴퓨터가 처리할 수 있는 형태로 모델링할 수 있다. • 문제에 알맞은 데이터 구조(리스트·2차원 리스트·딕셔너리)와 제어 구조(반복·조건의 중첩)를 선택하면 프로그램을 더 간결하고 효율적으로 만들 수 있다. • 프로그램은 테스트와 개선, 공유의 반복을 통해 더 정확하고 사용하기 좋은 산출물로 발전한다.		
핵심 개념	**[핵심 개념]** 문제 분해, 추상화(모델링), 알고리즘(순서도·의사코드), 리스트·2차원 리스트·딕셔너리, 문자열 처리(인덱싱·슬라이싱·in 연산자), 중첩 반복문·다중 조건문, 함수(매개변수·반환값), 입력 검증과 테스트 **[선행 개념]** 변수와 자료형(문자열), 연산자(산술·비교·논리), 표준 입출력(input/print), 기본 조건문(if)과 반복문(for/while), random 모듈		
핵심 질문	**익숙한 놀이(행맨 게임)의 규칙을, 컴퓨터가 이해하고 실행할 수 있는 알고리즘과 프로그램으로 어떻게 바꿀 수 있을까?** • 행맨 게임의 규칙을 어떤 작은 문제로 나누어야 할까? (분해) • 비밀 단어·맞힌 글자·틀린 글자·그림 단계처럼 게임에 꼭 필요한 데이터는 무엇이고, 어떤		

	구조에 담아야 할까? (추상화·데이터 구조) • 대문자·숫자·이미 입력한 글자처럼 어떤 입력에도 내 프로그램이 올바르게 동작한다는 것을 어떻게 확인할 수 있을까? (테스트·개선)
프로젝트 개요	단어 맞히기 놀이인 행맨(Hangman) 게임을 공통 소재로 「문제 발견 → 문제 구조화 → 컴퓨팅 설계 → 구현 → 테스트 및 개선 → 공유·확장」의 6단계를 8차시 동안 수행하는 프로젝트이다. 학생은 교사가 제공한 행맨 게임을 체험·분석하여 규칙을 입력–처리–출력으로 분해하고, 순서도·의사코드로 알고리즘을 설계한 뒤, 리스트(행맨 그림·단어 목록)·딕셔너리(주제별 단어와 뜻·점수)·2차원 리스트(난이도표·기록표·키보드판)와 중첩 제어 구조를 사용하여 파이썬으로 구현한다. 모둠은 행맨 게임을 확장하거나, 행맨에서 익힌 구조를 적용한 새 텍스트 게임(숫자야구, 끝말잇기, 영단어 퀴즈 등)을 기획할 수 있다. 이어서 반복되는 코드를 함수로 모듈화하고, 정상·경계·예외 입력(대문자, 숫자, 두 글자, 이미 입력한 글자 등)의 테스트 케이스로 오류를 찾아 개선하며, 완성한 게임을 서로 시연하고 피드백한다.
프로젝트 목표	1. 행맨 게임의 규칙을 입력·처리·출력 단계의 작은 문제로 분해하고 핵심 요소를 추상화하여 모델링할 수 있다. 2. 문제에 적합한 데이터 구조(리스트·2차원 리스트·딕셔너리)를 선택하여 단어·글자·기록 데이터를 저장·가공할 수 있다. 3. 중첩 반복문과 다중 조건문을 복합적으로 활용하고, 함수로 모듈화한 게임 프로그램을 작성할 수 있다. 4. 테스트 케이스를 설계하여 오류를 찾고 수정하며, 결과물을 공유하고 동료 피드백으로 개선할 수 있다.
수행평가 과제 (탐구 과제)	**[수행평가] 알고리즘 문제 프로그래밍하기 (실험·실습, 프로젝트 / 교사 관찰 및 기록)** • 과제: 행맨 게임을 확장(주제·난이도·뜻 힌트·기록 기능 등)하거나 행맨의 구조를 적용한 텍스트 게임 1종을 기획하여 문제 분해 → 설계 → 구현 → 테스트 과정을 활동지에 기록하고, 2차원 리스트·딕셔너리와 복합 제어 구조, 함수를 활용한 게임 프로그램(.py)을 완성하여 제출한다. • 평가 요소(배점): 문제 분해 및 추상화(20), 다차원 데이터 구조의 적용(25), 복합 제어 구조의 설계(20), 프로그램 구현 및 최적화(20), 산출물 완성도(15) – 기본점수 35점 • AI는 소스코드 작성·오류 수정 범위에서만 활용하며, 활용 과정을 활동지 서식에 기록한다 (개인정보 입력 금지).

Ⅱ. 차시별 6단계 수업 모형 적용 과정(총 8차시)



단계	차시	소단원 제목	교수-학습 내용 및 과정
문제 발견	1	**놀이 속 알고리즘 찾기 – 행맨 게임 체험과 프로젝트 주제 정하기**	• [동기] 교사와 칠판 행맨 한 판 → 교사가 제공한 행맨 게임(game_행맨_2_함수.py)을 직접 실행해 보기 • 행맨 게임의 규칙을 입력(사용자가 하는 일)-처리(컴퓨터가 판단하는 일)-출력(보여 주는 것)-종료 조건으로 정리하기 [활동지 1] • "컴퓨터는 숨겨진 단어와 맞힌 글자를 어떻게 기억하고 있을까?" 질문으로 코드 일부(행맨그림 리스트, 게임화면생성 함수)를 함께 읽어 보기 • 모둠(2~3명) 구성, 프로젝트 게임 선정(행맨 확장 또는 행맨 구조를 적용한 새 게임 기획), 수행평가 기준·AI 활용 규칙 안내
문제 구조화	2	**행맨 규칙 분해와 추상화 – 문제를 작은 문제로 나누기**	• 행맨 게임을 하위 문제(단어 고르기·화면 그리기·입력 받기와 검사·판정·종료 조건·다시 하기)로 분해하여 분해 트리 그리기 [활동지 2] • 추상화: 게임에 꼭 필요한 요소(비밀 단어, 맞힌 글자, 틀린 글자, 남은 기회, 그림 단계)와 버려도 되는 요소(분필 색, 교수대 모양, 진행자 말투 등) 구분하기 • 각 하위 문제에 필요한 데이터를 변수·리스트·딕셔너리 후보로 적어 보기 (예: 틀린 횟수 = 행맨그림 리스트의 인덱스) • 모둠별 분해 결과를 짧게 공유하고 교사 피드백 받기
컴퓨팅 설계	3~4	**알고리즘 설계 – 순서도·의사코드와 데이터 구조·함수 설계**	[3차시] 순서도·의사코드로 행맨 한 판의 흐름 설계, 반복(while)과 분기(if-else)의 위치와 두 가지 종료 조건(모두 맞힘 / 6번 틀림) 정하기 [3차시] 데이터 구조 선택: 행맨 그림·단어 목록 → 리스트, 주제별 단어와 뜻·점수 → 딕셔너리, 난이도표·게임 기록표·키보드판 → 2차원 리스트 [4차시] 함수 설계표 작성: 함수 이름·매개변수·반환값·역할 정하기(예: 모두맞췄는지(비밀단어, 맞춘단어) → True / False) https://wikidocs.net/476045 [활동지 3]

단계	차시	소단원 제목	교수–학습 내용 및 과정
			[4차시] 모둠 간 설계 검토(다른 모둠의 의사코드를 따라가며 빠진 경우 찾기)
구현	5~6	**게임 프로그램 구현 – 기본 버전에서 함수 버전으로**	[5차시] 리스트·반복문·조건문만으로 기본 버전 구현 (예시: game_행맨_1_기본.py, 보충: game_행맨_빈칸채우기.py) [5차시] 문자열 인덱싱·슬라이싱과 in 연산자로 밑줄 단어(_ _ a _) 화면을 만드는 원리 이해하기 [6차시] 하위 문제별로 코드를 함수로 분리하고 main() 구조로 정리(리팩터링) (예시: game_행맨_2_함수.py) [6차시] 딕셔너리·2차원 리스트를 적용하여 주제 선택·뜻 힌트·난이도·기록표 기능 추가 (예시: game_행맨_3_산출물예시.py) [6차시] 주석 달기, 변수 이름 정리, AI 활용 시 활용 기록표 작성 → 세부 지도안은 III 수업지도안(본시) 참조
테스트 및 개선	7	**테스트 케이스로 오류 찾고 개선하기**	• 오류의 종류(문법·실행·논리 오류) 확인, 정상·경계·예외 입력으로 테스트 케이스 표 작성 [활동지 4] • 다른 모둠과 프로그램을 바꾸어 테스트하고 실패 사례 기록 • 입력 검증 함수(strip, lower, len, in 검사, 이미 사용한 글자 검사)로 오류 수정 • 심화: 글자 빈도를 딕셔너리로 세어 '먼저 고를 글자' 전략과 확률 탐구, 기록표(2차원 리스트)를 중첩 반복으로 출력
공유 확장	8	**게임 시연과 코드 리뷰 – 성능 평가와 확장**	• 갤러리 워크: 모둠별 게임 시연, 다른 모둠이 직접 플레이하고 동료 피드백지 작성 [활동지 5] • 코드 리뷰: 데이터 구조·함수 분리·예외 처리 측면에서 잘된 점과 개선점 공유 • 확장 아이디어 토의: 파일 입출력으로 단어 목록 불러오기·기록 저장, 2인용(친구가 단어 입력) 모드, 난이도별 기회 수 조절, 클래스(객체)로 게임 구조화 • 최종 산출물(.py, 활동지, AI 활용 기록) 제출 및 프로젝트 성찰 쓰기

Ⅲ. 5~6차시 수업 지도안 (구현 단계 · 본시)

• 대상·차시: 고등학교 1학년 / 5~6차시(전체 8차시 중), 차시당 50분 / 6단계 수업 모형 중 ④ 구현 단계

• 차시 목표: [5차시] 의사코드를 리스트·반복문·조건문으로 옮겨 행맨 기본 버전을 완성할 수 있다. [6차시] 하위 문제를 함수로 분리하고, 딕셔너리·2차원 리스트로 기능을 확장할 수 있다.

• 준비물: 학생 노트북(파이썬 3·IDLE), 수업 PPT, game_행맨_1_기본.py · game_행맨_빈칸채우기.py · game_행맨_2_함수.py · game_행맨_3_산출물예시.py, [활동지 3](의사코드·함수 설계표), AI 활용 기록표

• 수업 전략: 짝 프로그래밍(드라이버·내비게이터 10분마다 교대), 의사코드 한 줄 ↔ 코드 한 덩어리 대응, 수준별 스캐폴딩(빈칸 채우기 → 기본 → 함수 → 확장)

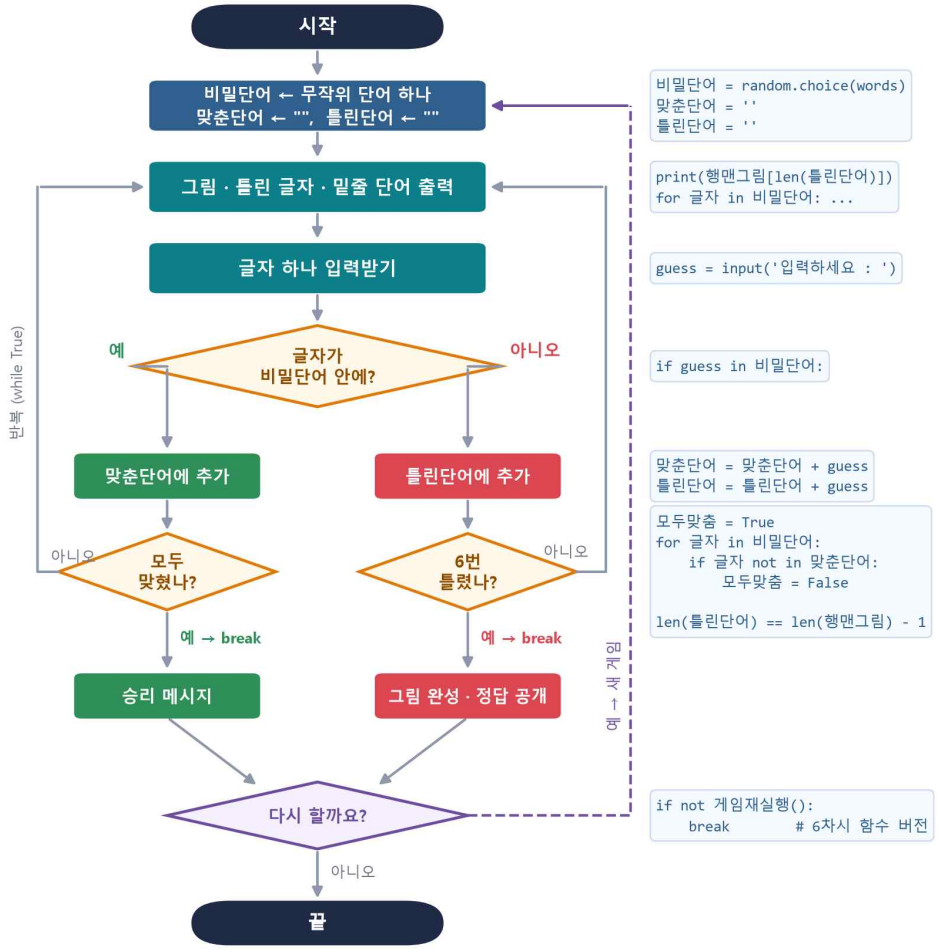


<그림 1> 5~6차시 수업 흐름과 시간 배분 (차시당 50분)

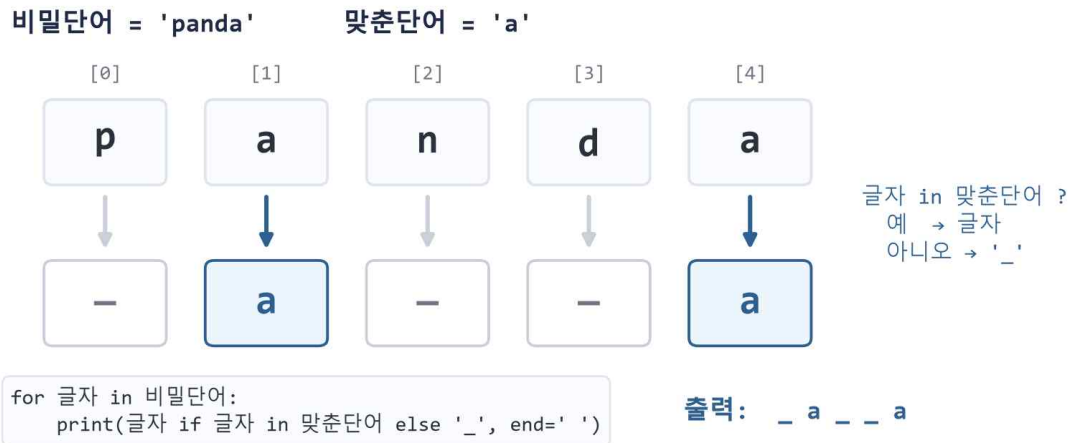
[5차시] 행맨 기본 버전 구현 – 의사코드를 리스트·반복문·조건문으로 옮기기

수업 흐름		핵심 질문	교수–학습 활동
열기	동기 유발 및 안내	"의사코드 한 줄은 파이썬 코드 몇 줄이 될까?"	**[동기 유발] (3분)** • 지난 차시 [활동지 3]의 의사코드를 화면에 띄우고, 완성된 행맨(game_행맨_2_함수.py)을 1판 시연 • "의사코드의 '밑줄 단어를 출력'은 파이썬으로 어떻게 쓸까?" 질문으로 설계와 코드 사이의 거리 느끼기 • 오늘의 산출물: 함수 없이 동작하는 행맨 기본 버전 (game_행맨_1_기본.py 수준)
			[학습 목표 및 활동 안내] (2분) • 의사코드를 리스트·반복문·조건문으로 옮겨 행맨 기본 버전을 구현할 수 있다. • 문자열 인덱싱과 in 연산자를 활용하여 밑줄 단어 화면을 만들 수 있다. • 활동 순서: 개념 확인 → 화면 출력 → 입력·판정 → 종료 조건 → 짝 점검 → 정리 (<그림 1>)
펼치기	개념 확인	"행맨을 만드는 데 필요한 파이썬 도구는 무엇일까?"	**[개념 확인] (5분)** • random.choice(리스트), 문자열 연결(+), in 연산자, len(), while True와 break를 짧은 예제로 확인 • 퀴즈: 'a' in 'panda' → ? / len('') → ? / 행맨그림

수업 흐름		핵심 질문	교수-학습 활동
			[len('ab')]는 몇 번째 그림? • 짝 역할 정하기: 드라이버(코드 입력)·내비게이터(의사코드 대조), 10분마다 교대
	탐구 활동	"그림과 밑줄 단어는 어떻게 화면에 나타날까?"	**[탐구 1: 화면 출력 구현] (10분)** • 행맨그림 리스트에서 len(틀린단어)번째 그림 출력 - 틀린 횟수가 곧 인덱스임을 확인 • for 글자 in 비밀단어: 반복과 in 검사로 밑줄 단어(_ a _ _ a) 출력 (<그림 3>) • 확인: 맞춘단어 = 'a'처럼 값을 직접 바꾸어 실행하며 출력 변화 관찰
		"입력한 글자는 어디에 기록해야 할까?"	**[탐구 2: 입력과 판정 구현] (10분)** • input()으로 글자를 받고 if guess in 비밀단어: 로 판정 • 맞으면 맞춘단어, 틀리면 틀린단어에 이어 붙이기(문자열 +) → 다음 반복에서 화면이 바뀌는지 확인 • 교사 순회: "틀린 글자를 기록하지 않으면 그림은 어떻게 될까?" 질문
		"게임은 언제, 어떻게 끝나야 할까?"	**[탐구 3: 종료 조건 구현] (8분)** • 종료 조건 ① 모두 맞힘: 모두맞춤 = True로 두고, for 반복 중 못 맞힌 글자가 있으면 False (깃발 변수) • 종료 조건 ② 6번 틀림: len(틀린단어) == len(행맨그림) - 1 이면 정답 공개 후 break • 순서도(<그림 2>)와 내 코드를 한 칸씩 짚어 가며 빠진 분기 확인
	점검 및 보충 심화	"내 코드는 의사코드의 모든 줄을 구현했을까?"	**[짝 점검] (4분)** • 짝과 코드를 바꾸어 의사코드 줄마다 대응하는 코드에 ✓ 표시, 승리·패배를 각 1번씩 실행하여 확인 • 오류 메시지가 나면 문법·실행 오류 위치를 함께 찾기 (7차시 테스트 활동 예고)
		"먼저 끝낸 모둠은 무엇을 더할 수 있을까?"	**[보충·심화] (4분) - 수준별 선택** • 보충: game_행맨_빈칸채우기.py의 빈칸 ①~⑩ 채우기, 의사코드 카드 순서 맞추기 • 심화: 남은 기회 표시하기, 단어 목록을 우리 모둠 주제로 바꾸기, 대문자 입력 처리(lower) 미리 해 보기
넓히기	**공유**	"기본 버전의 아쉬운 점은 무엇일까?"	**[공유 및 정리] (4분)** • 1~2개 모둠 시연, '가장 막혔던 줄'과 해결 방법 공유 • 같은 코드가 반복되거나 너무 길어진 곳 찾기 → 다음 차시 '함수로 정리하기' 예고 • 코드 저장(파일명 규칙 안내), AI를 활용했다면 AI 활용 기록표 작성



<그림 2> 행맨 한 판 순서도와 대응 코드 (5차시 탐구 1~3)

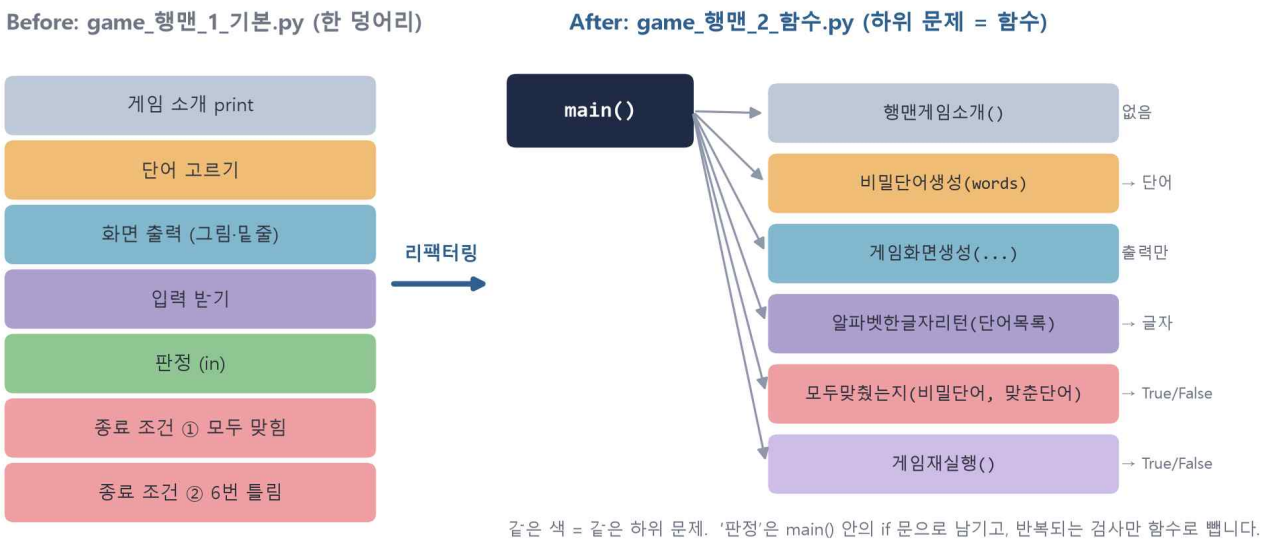


<그림 3> 문자열과 in 연산자로 밑줄 단어 만들기 (5차시 탐구 1)

[6차시] 함수로 정리하고 딕셔너리 · 2차원 리스트로 확장하기

수업 흐름		핵심 질문	교수–학습 활동
열기	동기 유발 및 안내	“규칙이 바뀌면 코드의 어디를 고쳐야 할까?”	**[동기 유발] (3분)** • 5차시 기본 버전을 띄우고 “주제를 고르는 기능을 넣으려면 어디를 고쳐야 할까?” 질문 → 한 덩어리 코드에서 찾기 어려움 체감 • 같은 동작을 함수로 나눈 game_행맨_2_함수.py와 나란히 비교 (<그림 4>)
			[학습 목표 및 활동 안내] (2분) • 하위 문제를 매개변수와 반환값이 있는 함수로 분리하여 main() 구조로 정리할 수 있다. • 딕셔너리와 2차원 리스트를 활용하여 주제 선택·뜻 힌트·난이도·기록표 기능을 추가할 수 있다. • 활동 순서: 개념 확인 → 함수 분리 → 딕셔너리 → 2차원 리스트 → 짝 점검 → 정리
펼치기	개념 확인	“함수는 무엇을 받고 무엇을 돌려줄까?”	**[개념 확인] (5분)** • def, 매개변수, return, 함수 호출, if __name__ == '__main__': main() 구조 확인 • [활동지 3]의 함수 설계표와 실제 함수(모두맞췄는지 → True/False) 연결하기 • 퀴즈: return이 없는 함수의 결과는? 함수 안에서 만든 변수를 밖에서 쓸 수 있을까?
	탐구 활동	“하위 문제 하나를 함수 하나로 바꾸어 볼까?”	**[탐구 1: 함수로 분리하기(리팩터링)] (10분)** • 종료 조건 ①을 모두맞췄는지(비밀단어, 맞춘단어)로, 화면 출력을 게임화면생성(...)으로 분리 • 입력 받기를 알파벳한글자리턴(단어목록)으로 분리하고 main()에서 호출 (<그림 4>) • 분리 후 반드시 실행하여 기능이 그대로인지 확인 (리팩터링 = 동작은 같고 구조만 개선)
		“주제별 단어와 뜻은 어떤 구조에 담으면 좋을까?”	**[탐구 2: 딕셔너리 적용] (10분)** • 단어사전 = {'동물': {'cat': '고양이', …}, '과일': {…}}로 주제별 단어와 뜻 저장 (<그림 5>) • 주제를 번호로 고르고 단어사전[주제]에서 비밀 단어 선택, 단어사전[주제][비밀단어]로 뜻 힌트 출력 • 점수 = {'승': 0, '패': 0}에 결과 누적 (점수[결과] += 1)
		“여러 판의 기록을 표처럼 저장하려면?”	**[탐구 3: 2차원 리스트 적용] (8분)** • 난이도표 = [['쉬움', 3, 5, True], …]에서 난이도표[행][열]로 글자 수 범위 읽기 • 판이 끝날 때마다 기록표.append([판, 주제, 난이도, 단어, 결과, 틀림]) 후 중첩 반복문으로 표 출력

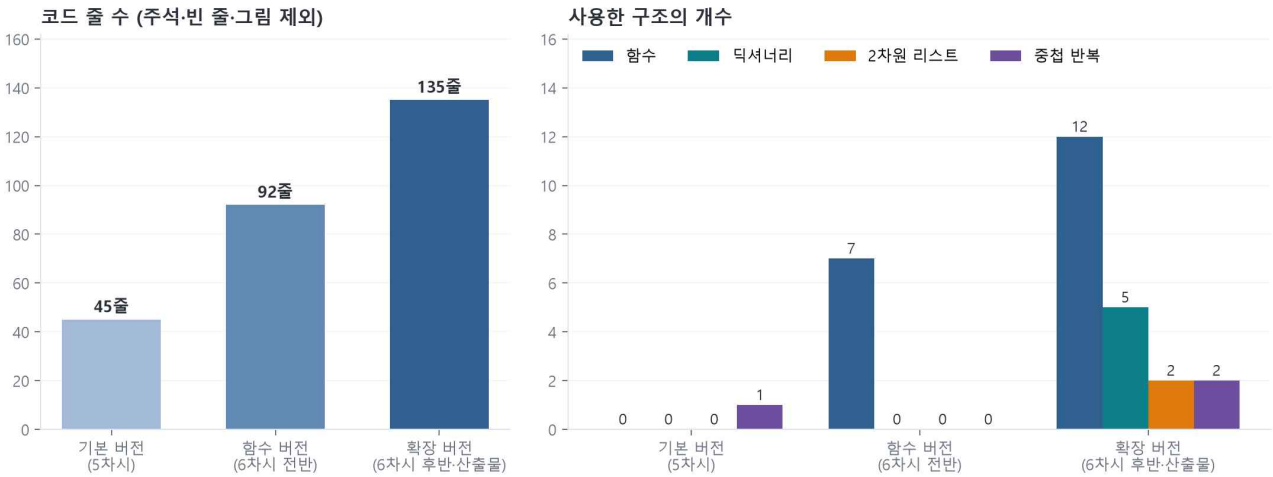
수업 흐름		핵심 질문	교수–학습 활동
			• 교사 순회: 행·열 순서를 바꾸어 쓴 인덱싱 오류를 함께 추적
	점검 및 보충 심화	“함수로 나눈 뒤에도 게임이 똑같이 동작할까?”	**[짝 점검] (4분)** • 리팩터링 전후를 같은 입력으로 실행하여 결과 비교, 함수 이름·매개변수·반환값이 설계표와 일치하는지 확인 • 수행평가 체크: 2차원 리스트·딕셔너리를 1개 이상 목적에 맞게 사용했는가
		“2차원 리스트로 또 무엇을 표현할 수 있을까?”	**[보충·심화] (4분) – 수준별 선택** • 보충: game_행맨_3_산출물예시.py를 실행하고 단어사전에 새 주제 추가하기 • 심화: 키보드판 2차원 리스트를 중첩 반복으로 출력하여 맞힌 글자·틀린 글자 표시, 난이도별 기회 수 조절
넓히기	공유	“우리 게임은 얼마나 성장했을까?”	**[공유 및 정리] (4분)** • 버전별 함수·자료구조 수 변화(<그림 6>)를 보며 오늘 추가한 구조를 모둠별로 한 문장 발표 • 다음 차시 예고: 7차시 ‘테스트 케이스로 오류 찾고 개선하기’ – 친구가 이상한 값을 넣어도 버틸까? • 코드 저장, AI를 활용했다면 AI 활용 기록표 작성



<그림 4> 리팩터링 전후 코드 구조 – 하위 문제 하나 = 함수 하나 (6차시 탐구 1)



<그림 5> 딕셔너리 단어사전과 2차원 리스트 난이도표·기록표 (6차시 탐구 2~3)



<그림 6> 버전별 코드 구조 변화 – 제공 예시 코드 3종을 실제로 분석한 결과 (6차시 정리)

IV. 학생 활동지 및 학습 자료

[활동지 1] 1차시 – 행맨 게임 속 알고리즘 찾기

모둠: _____ 이름: _____ (※ 수행평가 활동지 제출용)

게임	입력 (사용자가 하는 일)	처리 (컴퓨터가 판단하는 일)	출력 (보여 주는 것)	종료 조건
행맨	알파벳 한 글자	비밀 단어에 그 글자가 있는지 판단, 맞힌 글자·틀린 글자 기록	행맨 그림, 밑줄 단어(_ _ a _), 틀린 글자	단어를 모두 맞힘 또는 6번 틀림
숫자 야구				
우리 모둠 게임				

Q. 행맨 게임에서 반복·조건·리스트·문자열은 각각 어디에 쓰였는가? _____

[활동지 2] 2차시 – 문제 분해와 추상화

하위 문제	하는 일	필요한 데이터(변수·리스트·딕셔너리)	담당
① 입력 받기·검사	알파벳 한 글자인지, 이미 입력한 글자인지 확인하고 아니면 다시 묻기	guess, 틀린단어 + 맞춘단어	
② 판정하기			
③ 화면 그리기(그림·밑줄)			
④ 종료 조건			
⑤ 다시 하기·기록			

추상화 – 꼭 필요한 것: _____ 빼도 되는 것: _____

[활동지 3] 3~4차시 – 알고리즘 설계

① 의사코드(또는 순서도)로 게임 한 판의 흐름을 적어 보세요. (예시)

```
비밀단어 ← 단어 목록에서 무작위로 하나,  맞춘글자 ← "",  틀린글자 ← ""
반복:
    그림[틀린글자 수], 틀린 글자, 밑줄 단어를 출력
    글자 ← 올바른 알파벳 한 글자를 입력받기
    만약 글자가 비밀단어 안에 있으면: 맞춘글자에 글자 추가
        만약 비밀단어의 모든 글자가 맞춘글자에 있으면: "승리" 출력 후 반복 종료
    아니면: 틀린글자에 글자 추가
        만약 틀린글자 수 = 6 이면: "실패, 정답은 ○○" 출력 후 반복 종료
다시 할지 묻기 → yes면 처음부터
```

② 함수 설계표

함수 이름	매개변수	반환값	하는 일
알파벳한글자리턴	단어목록(사용한 글자)	알파벳 한 글자	올바른 글자가 입력될 때까지 다시 묻는다
모두맞췄는지	비밀단어, 맞춘단어	True / False	비밀 단어의 글자를 모두 맞혔는지 확인한다

③ 데이터 구조 선택: 리스트( ), 딕셔너리( ), 2차원 리스트( )

[활동지 4] 7차시 – 테스트 케이스와 개선

번호	구분	입력값	예상 결과	실제 결과	통과	수정 내용
1	정상	a (단어에 있는 글자)	해당 자리에 글자 공개, 기회 유지			
2	정상	z (단어에 없는 글자)	틀린 글자에 추가, 그림 1단계 진행			
3	경계	마지막 남은 글자	단어 완성, 승리 메시지 후 종료			
4	경계	6번째 틀린 글자	그림 완성, 정답 공개 후 종료			
5	예외	A (대문자)	소문자 a로 바꾸어 판정			
6	예외	1 / ab / ㄱ / (빈 입력)	“알파벳 한 글자” 다시 입력 요청, 기회 유지			
7	예외	이미 입력한 글자	“이미 입력한 글자” 안내, 기회 유지			

※ 예시는 행맨 게임 기준. 각 모둠은 자기 게임에 맞게 고쳐 쓴다.

[AI 활용 기록표] (수행평가 유의사항에 따른 필수 기록)

차시	활용 목적 (코드 작성/오류 수정)	AI에 입력한 내용 (개인정보 입력 금지)	AI 답변 요약	내가 확인·수정한 내용

[활동지 5] 8차시 – 게임 시연 동료 피드백

평가 항목	점수(1~3)	의견
규칙이 이해하기 쉽고 게임이 재미있다		
이상한 값(대문자·숫자·중복 글자)을 입력해도 멈추지 않는다		
코드가 함수로 잘 나뉘어 있고 주석이 있다		
리스트·딕셔너리·2차원 리스트를 알맞게 썼다		

프로젝트 성찰: 가장 어려웠던 점과 해결 방법 / 다음에 더 해 보고 싶은 기능

[학습 자료]

- 수업용 프레젠테이션: 행맨게임_알고리즘_수업.pptx
- 기본 버전 예시 코드(리스트·반복문·조건문): game_행맨_1_기본.py (7차시 테스트용으로 입력 검사를 일부러 생략)
- 함수 버전 예시 코드(함수·매개변수·반환값·main 구조, 첨부 행맨 코드 기반): game_행맨_2_함수.py
- 산출물 예시 코드(딕셔너리·2차원 리스트·중첩 반복): game_행맨_3_산출물예시.py
- 보충·심화 자료: game_행맨_빈칸채우기.py(정답은 교사용 폴더), 행맨_심화_글자빈도.py, 행맨_학생활동지.docx
- 실습 환경: 파이썬 3 (IDLE 또는 학교 실습 환경) ※ 학교 실습실 환경 확인 필요

V. 평가 계획

평가 요소	성취수준			평가 역량	지도 유의사항 및 피드백
	상	중	하		
문제 분해 및 추상화 (20점)	게임 규칙을 입력·처리·출력으로 정확히 분해하고, 하위 문제 간 관계를 논리적으로 모델링함	게임 규칙을 입력·처리·출력으로 분해하였으나 하위 문제 간 관계 모델링이 부분적임	도움을 받아 게임 규칙을 일부 분해하고 모델링함	컴퓨팅 사고력 (분해·추상화)	분해 트리에 빠진 하위 문제(두 가지 종료 조건, 입력 검사 등)를 질문으로 짚어 줌
다차원 데이터 구조의 적용 (25점)	2차원 리스트·딕셔너리를 목적에 맞게 선택하여 효율적으로 문제를 해결함	2차원 리스트·딕셔너리를 활용하여 문제를 해결하거나 일부 문제를 해결함	도움을 받아 활용하거나 다차원 데이터 구조를 활용하지 않음	컴퓨팅 사고력 (자료 표현)	주제별 단어 딕셔너리·기록표 예시를 제시하고, 인덱싱 오류(행·열 순서)를 함께 추적
복합 제어 구조의 설계 (20점)	중첩 반복문과 다중 조건문을 설계·처리하여 목적에 맞는 프로그램을 작성함	중첩 반복문과 다중 조건문을 부분적으로 설계하고 도움을 받아 작성함	중첩 반복·다중 조건을 부분적으로 이해하고 도움을 받아 일부 작성함	컴퓨팅 사고력 (알고리즘)	반복의 종료 조건과 분기 순서를 의사코드로 먼저 확인하게 함
프로그램 구현 및 최적화 (20점)	예외 입력까지 완벽하게 처리하고, 함수로 모듈화한 효율적인 프로그램을 작성함	예외 상황을 일부 처리하며 도움을 받아 목적에 맞는 프로그램을 작성함	예외 상황을 일부 처리하며 도움을 받아 부분적으로 작성함	컴퓨팅 사고력 (자동화) AI 활용 윤리	테스트 케이스 표를 근거로 피드백, AI 활용 기록과 코드 설명 일치 여부 확인
산출물 완성도 (15점)	프로그램 산출물을 정확한 위치에 저장하고 제출함	도움을 받아 해당 위치에 저장하고 제출함	도움을 받아 부분적으로 저장하고 제출함	디지털 문화 소양 (협업·공유)	제출 위치·파일명 규칙 사전 안내, 미제출 학생 개별 확인

※ 상: 성취수준 A~B, 중: C~D, 하: E에 해당하며, 세부 배점은 「수행평가 세부계획(알고리즘 문제 프로그래밍하기)」 채점 기준표를 따른다. 기본점수(참여하였으나 산출물이 없는 경우) 35점.
※ 평가 역량 칸의 역량명은 정보과 교과 역량을 참고하여 작성함(학교 계획과 대조 필요).

VI. 학생 산출물 예시

※ 아래는 예시

1. 문제 분해 결과 (행맨 확장, ○○모듬 예시)

하위 문제	입력	처리	출력	데이터 구조
글자 입력	알파벳 한 글자	공백 제거·소문자 변환 후 한 글자·알파벳·중복 검사, 아니면 다시 묻기	안내 문구	문자열(사용한 글자)
판정·화면	입력 글자	비밀 단어에 있으면 맞춘단어, 없으면 틀린단어에 추가	그림, 밑줄 단어, 키보드판	리스트 행맨그림, 2차원 리스트 키보드판
단어·난이도	주제·난이도 번호	주제 딕셔너리에서 난이도표의 글자 수에 맞는 단어 고르기	글자 수, 뜻 힌트	딕셔너리 단어사전, 2차원 리스트 난이도표
종료·기록	한 판 결과	모두 맞힘 또는 6번 틀림이면 종료, 점수·기록 추가	기록표, 승패 수	딕셔너리 점수, 2차원 리스트 기록표

2. 프로그램 코드 (game_행맨_3_산출물예시.py, 행맨 그림 부분은 생략)

```
# 행맨 게임 (산출물 예시) - 딕셔너리(주제별 단어 · 뜻, 점수), 2차원 리스트(난이도표 · 키보드판 · 기록표), 중첩 반복, 함수
import random

행맨그림 = [r'''...''', ...]          # 0~6단계 그림 7개 (game_행맨_1_기본.py와 같음, 생략)
최대틀림 = len(행맨그림) - 1                    # 6번 틀리면 끝

# 딕셔너리 안의 딕셔너리: 주제 → {영어 단어: 한글 뜻}
단어사전 = {
    '동물': {'cat': '고양이', 'tiger': '호랑이', 'rabbit': '토끼', 'zebra': '얼룩말',
             'monkey': '원숭이', 'turtle': '거북', 'penguin': '펭귄', 'elephant': '코끼리'},
    '과일': {'apple': '사과', 'grape': '포도', 'lemon': '레몬', 'banana': '바나나',
             'cherry': '체리', 'orange': '오렌지', 'pineapple': '파인애플', 'strawberry': '딸기'},
    '정보': {'list': '리스트', 'loop': '반복', 'input': '입력', 'python': '파이썬',
             'output': '출력', 'variable': '변수', 'function': '함수', 'algorithm': '알고리즘'},
}
```

```
# 2차원 리스트: 난이도표[행] = [이름, 최소 글자 수, 최대 글자 수, 뜻 힌트 제공]
난이도표 = [['쉬움', 3, 5, True],
           ['보통', 5, 6, False],
           ['어려움', 7, 20, False]]

# 2차원 리스트: 키보드판 (행 = 키보드 줄, 열 = 글자)
키보드판 = [list('qwertyuiop'),
           list('asdfghjkl'),
           list('zxcvbnm')]
```

```
def 행맨게임소개():
    print('*' * 40)
    print(' 행맨 게임 - 주제와 난이도를 골라 보세요')
    print(f' 알파벳 한 글자씩 입력, {최대틀림}번 틀리면 끝!')
    print('*' * 40)


def 번호선택(안내문, 선택지):
    """선택지(리스트)를 번호와 함께 보여 주고, 올바른 번호가 들어올 때까지 다시 묻는다"""
    for i in range(len(선택지)):
        print(f'  {i + 1}. {선택지[i]}')
    while True:
        text = input(안내문 + ' > ').strip()
        if text.isdigit() and 1 <= int(text) <= len(선택지):
            return int(text) - 1                # 인덱스로 돌려준다
        print(f'1~{len(선택지)} 사이의 번호를 입력하세요.')
```

```
def 비밀단어생성(주제, 난이도):
    """주제의 단어 중 난이도 글자 수에 맞는 단어를 무작위로 고른다"""
    최소, 최대 = 난이도[1], 난이도[2]
```

```
    후보 = []
    for 단어 in 단어사전[주제]:
        if 최소 <= len(단어) <= 최대:
            후보.append(단어)
    if len(후보) == 0:                          # 맞는 단어가 없으면 전체에서 고른다
        후보 = list(단어사전[주제])
    return random.choice(후보)
```

```
def 밑줄단어(비밀단어, 맞춘단어):
    """맞힌 글자는 보여 주고 나머지는 _ 로 바꾼 문자열을 돌려준다"""
    결과 = ''
    for 글자 in 비밀단어:
        if 글자 in 맞춘단어:
            결과 = 결과 + 글자 + ' '
        else:
            결과 = 결과 + '_ '
    return 결과
```

```
def 키보드출력(틀린단어, 맞춘단어):
    """2차원 리스트 키보드판을 중첩 반복문으로 출력한다 (맞힘: 대문자, 틀림: ·)"""
    for 행 in range(len(키보드판)):
        print(' ' * 행, end='')                 # 줄마다 한 칸씩 들여쓰기
        for 열 in range(len(키보드판[행])):
            글자 = 키보드판[행][열]
            if 글자 in 맞춘단어:
                print(글자.upper(), end=' ')
            elif 글자 in 틀린단어:
                print('·', end=' ')
            else:
                print(글자, end=' ')
        print()
```

```
def 게임화면생성(틀린단어, 맞춘단어, 비밀단어):
    print(행맨그림[len(틀린단어)])
    print(f'남은 기회: {최대틀림 - len(틀린단어)}번   틀린 글자: {" ".join(틀린단어)}')
    키보드출력(틀린단어, 맞춘단어)
    print()
    print('단어: ' + 밑줄단어(비밀단어, 맞춘단어))


def 알파벳한글자리턴(사용한글자):
    """공백 제거·소문자 변환 후, 사용하지 않은 알파벳 한 글자가 들어올 때까지 다시 묻는다"""
    while True:
        guess = input('알파벳 한 글자 > ').strip().lower()
        if len(guess) != 1:
            print('한 글자만 입력해 주세요.')
        elif guess not in 'abcdefghijklmnopqrstuvwxyz':
            print('알파벳(a~z)만 입력해 주세요.')
        elif guess in 사용한글자:
            print('이미 입력한 글자입니다. 다른 글자를 고르세요.')
        else:
            return guess
```

```
def 모두맞췄는지(비밀단어, 맞춘단어):
    for 글자 in 비밀단어:
        if 글자 not in 맞춘단어:
            return False
    return True
```

```
def 한판진행(주제, 난이도):
    """한 판을 진행하고 [비밀단어, 결과, 틀린 횟수]를 돌려준다"""
    비밀단어 = 비밀단어생성(주제, 난이도)
    맞춘단어, 틀린단어 = '', ''
    print(f'₩n[{주제} / {난이도[0]}] {len(비밀단어)}글자 단어입니다.')
    if 난이도[3]:
        print('뜻 힌트: ' + 단어사전[주제][비밀단어])

    while True:
        게임화면생성(틀린단어, 맞춘단어, 비밀단어)
        guess = 알파벳한글자리턴(틀린단어 + 맞춘단어)
        if guess in 비밀단어:
```

```
            맞춘단어 = 맞춘단어 + guess
            if 모두맞췄는지(비밀단어, 맞춘단어):
                print(f'₩n축하합니다! 정답은 {비밀단어} ({단어사전[주제][비밀단어]})')
                return [비밀단어, '승', len(틀린단어)]
        else:
            틀린단어 = 틀린단어 + guess
            if len(틀린단어) == 최대틀림:
                print(행맨그림[최대틀림])
                print(f'₩n아쉬워요! 정답은 {비밀단어} ({단어사전[주제][비밀단어]})')
                return [비밀단어, '패', len(틀린단어)]
```

```
def 기록표출력(기록표):
    """2차원 리스트 기록표를 중첩 반복문으로 표처럼 출력한다"""
    제목줄 = ['판', '주제', '난이도', '단어', '결과', '틀림']
    for 행 in [제목줄] + 기록표:
        for 칸 in 행:
            print(str(칸).ljust(10), end='')
        print()


def 게임재실행():
    return input('다시 게임하실래요? (y/n) > ').strip().lower().startswith('y')
```

```
def main():
    행맨게임소개()
    점수 = {'승': 0, '패': 0}                  # 딕셔너리: 결과별 횟수
    기록표 = []                                 # 2차원 리스트: [[판, 주제, 난이도, 단어, 결과, 틀림], ...]
    주제목록 = list(단어사전.keys())
    난이도이름 = [행[0] for 행 in 난이도표]

    while True:
        주제 = 주제목록[번호선택('주제 번호', 주제목록)]
        난이도 = 난이도표[번호선택('난이도 번호', 난이도이름)]
        단어, 결과, 틀림 = 한판진행(주제, 난이도)
        점수[결과] += 1
        기록표.append([len(기록표) + 1, 주제, 난이도[0], 단어, 결과, 틀림])
        if not 게임재실행():
            break

    print('₩n===== 게임 기록표 =====')
    기록표출력(기록표)
    print(f'총 {len(기록표)}판  승: {점수["승"]}  패: {점수["패"]}')


if __name__ == '__main__':
    main()
```

3. 실행 결과 (일부)

```
  1. 동물   2. 과일   3. 정보
주제 번호 > 2          (난이도 번호 > 1 : 쉬움)
[과일 / 쉬움] 5글자 단어입니다.
뜻 힌트: 포도
알파벳 한 글자 > A
단어: _ _ a _ _
알파벳 한 글자 > a
이미 입력한 글자입니다. 다른 글자를 고르세요.
알파벳 한 글자 > x
남은 기회: 5번   틀린 글자: x
q w E r t y u i o p
 A s d f g h j k l
  z · c v b n m
단어: _ _ a _ e
   … (중략) …
축하합니다! 정답은 grape (포도)
===== 게임 기록표 =====
판        주제      난이도     단어      결과      틀림
1         과일      쉬움       grape     승        1
2         동물      보통       rabbit    승        4
```

총 2판 승: 2 패: 0

※ 비밀 단어는 무작위로 정해지므로 실행할 때마다 결과가 달라진다. (그림 출력은 생략)

4. 테스트 케이스 결과 (일부)

입력값	구분	예상 결과	실제 결과(수정 전)	수정 내용	수정 후
A	예외	a로 인정하여 판정	틀린 글자로 처리(실패)	input().lower() 적용	통과
ab	예외	다시 입력 요청	그림 2단계 진행(실패)	len(guess) != 1 검사 추가	통과
ㄱ	예외	다시 입력 요청	isalpha() 통과(실패)	in 'abc…z' 검사로 변경	통과
a␣(뒤 공백)	예외	‘a’로 인정	한 글자 아님(실패)	.strip() 적용	통과

5. AI 활용 기록 (예시)

활용 목적	AI에 입력한 내용	AI 답변 요약	내가 확인·수정한 내용
오류 수정	“파이썬에서 ‘ㄱ’.isalpha()가 True가 되는 이유와 영어 알파벳만 검사하는 방법”	한글도 문자(alpha)로 취급됨, "abcdefghijklmnopqrstuvwxyz"에 in으로 검사하는 방법 제시	알파벳한글자리턴()에 in 검사 적용 후 테스트 케이스 6번 재실행하여 통과 확인

Ⅶ. 교사 수업 설계 및 운영 유의사항

1. 수업 설계 측면

• 행맨 게임은 학생에게 친숙하고 규칙이 단순하여 '문제 발견'의 진입 장벽을 낮춘다. 단, 게임을 '만드는 것'보다 '규칙을 분해·모델링하는 과정'이 목표임을 차시마다 강조한다.

• 기본 버전(리스트·반복·조건) → 함수 버전(제공 행맨 코드) → 2차원 리스트·딕셔너리 확장 순서로 단계적 스캐폴딩을 제공한다. 제공 예시 코드는 정답이 아니라 출발점이며, 모둠별로 주제·난이도·힌트·기록 등 기능을 바꾸거나 추가하도록 안내한다.

• 수행평가 채점 기준(다차원 데이터 구조 25점)과 연결되도록, 함수 버전에 없는 2차원 리스트 활용(난이도표, 기록표, 키보드판 등)을 설계 단계(3~4차시)에서 반드시 계획하게 한다.

• 기본 버전(game_행맨_1_기본.py)은 대문자·숫자·두 글자·중복 글자 입력을 검사하지 않도록 의도적으로 개선 여지를 남겨 두었으므로, 7차시 테스트 활동의 소재로 활용한다.

2. 수업 운영 측면

• 모둠은 2~3명으로 구성하고, 드라이버(코드 입력)·내비게이터(검토) 역할을 차시마다 교대하여 무임승차를 줄인다. 수행평가는 개인 산출물 기준이므로 모둠 활동 후 개인별 코드를 완성·제출하게 한다.

• 행맨 그림에는 역슬래시(\)가 들어 있어 일반 문자열로 쓰면 파이썬 3.12 이상에서 경고(SyntaxWarning)가 나므로, 예시 코드는 r'''…''' (raw 문자열)로 작성하였다. 학생이 그림을 직접 바꿀 때도 같은 방법을 안내한다.

• 진도가 빠른 학생에게는 심화 과제(글자 빈도 전략, 키보드판·기록표 2차원 리스트, 파일 입출력으로 단어 목록 불러오기)를, 느린 학생에게는 빈칸 채우기형 코드(game_행맨_빈칸채우기.py)와 의사코드를 제공한다.

• 교사 순회 시 정답 코드를 바로 알려 주기보다 "이 반복은 언제 끝나니?", "이미 입력한 글자를 또 넣으면?"과 같은 질문으로 피드백한다.

3. 생성형 AI 활용 및 평가 유의사항

• AI는 소스코드 작성·오류 수정 범위에서만 활용할 수 있으며, 학번·이름·학교명 등 개인정보를 입력하지 않도록 매 차시 안내한다.

• AI 활용 시 활동지의 AI 활용 기록표를 작성하게 하고, 제출 코드에 대해 교사가 질의("이 줄은 무슨 일을 하니?")하여 설명하지 못하는 부분은 채점에서 제외될 수 있음을 사전에 안내한다.

• 수행평가는 수업 시간 내에 진행하고, 수업 종료 후 활동지를 제출받는다. 과정 중심 평가로서 교사 관찰 기록을 함께 남긴다.

• 결시·학적변동 학생 처리 및 기타 사항은 학교 학업성적관리규정과 교과협의회 결정에 따른다.

4. 확인이 필요한 사항 (작성자 메모)

• 경남형 AI 디지털 역량의 공식 요소명, 교육과정 핵심 아이디어 원문 문구는 확인 후 Ⅰ장에 보완 필요

• 수업 일시, 학급 수, 실습실 환경(파이썬 버전·IDE·노트북 수)은 미정으로 비워 둠, 영단어 목록은 학생 영어 수준에 맞게 조정 필요